runbook-gcp

Runbook — installing the infrastructure

Two paths. **A** works today on a laptop with no GCP account. **B** stands the real thing up on GCP with Terraform.

> **Update 2026-08-22.** `docs/PLAN-CHANGES-22082026.md` has landed: the Target System confirmation stream is wired. The mock publishes one `{runId, accountId, accountKey, confirmedAt}` event per accepted write (HTTP 201) to a second Kafka topic (`target-system-confirmations`); recon reads it with a fresh per-run consumer group, set-differences the keys against `account_target` , and fails the run on an unconfirmed row. The acceptance count is now 9 (criterion 9 checks the external claim). When the confirmation bootstrap is empty the path skips cleanly, so a no-Kafka run stays green.
>
> **Update 2026-08-21.** `docs/PLAN-CHANGES-21082026.md` has landed: the engine has two dispositions (`WRITTEN / REJECTED`), one full snapshot per run (no initial/delta, no window), and homogeneous TDS definitions. `make run-delta` and the window plumbing are gone; `make run` (alias `run-initial`) is the only run target, and the acceptance count is 8, not 10. Historical evidence under `docs/evidence/` and past-failure narratives below record runs under the old four-door/delta model and are left as-is.

A. Local stack (no billing, no GCP account)

Everything the prototype demonstrates runs here.

Prerequisites

Tool	Version verified	Notes
podman	4.9.3	**not docker** — see the warning below
podman-compose	1.0.6	`pip install --user podman-compose`
Python	3.11 (pinned by `uv`)	host 3.14 is too new for Beam/Airflow
`uv`	any	creates the pinned venv
Java	17-25	Maven (for 17 use `--release 17`)
GnuPG	2.4.4	the extraction lane's PGP round-trip
Node	20+	Dataform CLI, via `npx`

> **Use podman, not docker.** On this machine `/usr/local/bin/docker-compose` is a genuine Docker Compose binary that talks to `/var/run/docker.sock` , and there is no Docker daemon — it fails with

> `permission denied while trying to connect to the docker API`. All tooling here drives podman directly.
>
> If podman fails with `creating a temporary directory: ... no such file or directory`, its storage tree is incomplete. `make up` now creates it, or do it by hand: `mkdir -p ~/.local/share/containers/storage/tmp`.

Steps

```
        make bootstrap     # Python 3.11 venv, .env from .env.example, throwaway PGP keypair
        make up            # podman-compose: fake-gcs, bigquery-emulator, redpanda, target-system-mock
        make init-infra    # buckets, BigQuery datasets + tables, Kafka topic
        make verify-stack  # exercises each service, not just its port

        make run-initial   # full E → T+R → L run
        make verify        # assert all acceptance criteria
```

Verifying the negative path (criterion 9 can go red)

Criterion 9 ("every TARGET row confirmed by Target System") is the only criterion that checks an *external* system's own claim, so it is the one worth proving can fail. The mock exposes a one-shot suppression hook for that:

```
        # 1. Arm the next 201 to skip its confirmation publish (one-shot, consumed on the next accept).
        curl -X POST http://localhost:8080/__admin/suppress-next-confirmation

        # 2. Run the pipeline. Recon will fail with:
        #    RECONCILIATION FAILED: target system reconciliation GAP — 1 of N TARGET rows unconfirmed.
        make run           # exits non-zero

        # 3. `make verify` against that run reports 8/9 — criterion 9 names the unconfirmed account_key.
        make verify

        # 4. Reset the mock so the next run is clean.
        curl -X POST http://localhost:8080/__admin/reset
```

A clean run after the reset passes 9/9. The `target-system-confirmations` topic only exists when Kafka is enabled; a no-Kafka run reports criterion 9 as *skipped* (not failed), so `make verify` stays green without it.

Optional extras:

```
make up-airflow       # Cloud Composer stand-in on :8081 (admin/admin) — heavy, ~1 GB image
make verify-project2  # the zero-engine-diff extensibility proof
make test             # unit tests (Python + Java)
```

Building the Java apps (Maven, JDK 25)

The five Java apps build with Maven (`pom.xml` + `apps/*/pom.xml`), targeting **JDK 25** via `maven.compiler.release`.

```
make java-build     # self-contained jars at apps/<app>/target/<app>.jar
make test           # pytest + mvn test
```

Maven's shade plugin leaves the jars where `local/scripts/run_pipeline.py` invokes them with `java -jar`. A full `run-initial` on those jars passes all 9 acceptance criteria, locally and against real GCP (see `evidence/gcp-run-20260804/`).

JDK note. `maven.compiler.release` is 25, so an older JDK fails the build outright rather than degrading. The runtime images ship a matching JRE 25 (`Dockerfile.javaapp`, `Dockerfile.toolbox`, `apps/target-system-mock/Dockerfile`), so class-file version and runtime agree by construction. If `mvn` reports `release version 25 not supported`, `JAVA_HOME` points at an older or partial JDK — unset it or point it at a full JDK 25.

Why Airflow is a separate compose file. podman-compose 1.0.6 ignores `profiles:`, so a profiled service starts anyway. Splitting it into `local/docker-compose.airflow.yml` is the only way to keep it genuinely opt-in.

What is emulated

Real service	Local stand-in	Fidelity gap
GCS	`fsouza/fake-gcs-server`	no IAM, no lifecycle rules, no CMEK
BigQuery	`goccy/bigquery-emulator`	no load jobs, no partitioning, SQL subset. Returns **HTTP 500 "already created"** where real BQ returns 409 — the client adapter handles it
Dataflow	Beam **DirectRunner**	no autoscaling, no shuffle service, single process
Cloud Composer	Airflow 2.10 standalone, SequentialExecutor	sqlite, one task at a time
Managed Kafka	redpanda, PLAINTEXT	no SASL/OAUTHBEARER
Dataform	real `dataform compile` + a local SQL executor	the *models* are real; only the executor differs
Target System	`apps/target-system-mock`	injects 429/503 on purpose so retries are exercised; publishes one confirmation per accepted write to redpanda locally / Managed Kafka on GCP

B. Real GCP with Terraform

B0. Status — applied and run

> **Update 2026-08-23 — the confirmation stream is proven on GCP.** A full Composer DAG pass under `-var=enable_composer=true -var=enable_kafka=true` ran all 8 tasks green and closed acceptance criterion 9 from *live* Managed Kafka confirmations (50 of 50 TARGET rows confirmed, 0 unconfirmed) — see `evidence/gcp-composer-20260823/`. That pass also found and closed five defects that only real GCP could surface: the pod env-var gap (§B8.4), the Python `oauth_cb` signature/priming bug, the Managed Kafka token *encoding* (a bare access token is not accepted — see §B8.5), kafka-clients 3.7.1 being unusable for OAUTHBEARER on JDK 25, and `make build-images` never building the Cloud Run mock at all. The single item the plan listed as out of scope — a live criterion-9 green — is therefore now in scope and done; only criterion 4's *optional Kafka half* stays unverifiable from a laptop, for the host-reachability reason below.

This Terraform has been applied, repeatedly. The environment has been built from an empty project and torn down several times. The most recent full DAG pass is `evidence/gcp-composer-20260820/`: the migration DAG green on Cloud Composer 2, all 8 tasks, with the three Beam stages running as real Dataflow jobs and the loader and recon pods under their own least-privilege service accounts. That pass also closed the BigQuery streaming-buffer defect — `file_processor` now counts before it deletes, so a first run of a brand-new run id no longer fails a DML statement that matches zero rows (see `evidence/gcp-composer-20260820/02-failure-streaming-buffer.md`).

The two-door simplification (WRITTEN/REJECTED, one snapshot per run, homogeneous TDS) and the "Vault Core" → "Target System" rename landed after that pass (commits `a517b27` → `af96295`). The current HEAD was re-verified 2026-08-22 by `make smoke-gcp` 8/8 green against real GCS/BigQuery — see `evidence/gcp-smoke-20260822/`. That re-verification exercised the changed engine logic and naming; the DAG path's least-priv-SA + DataflowRunner plumbing was unchanged by the rename and needed no re-proof. The 2026-08-22 pass also surfaced a runbook gap — the DAG trigger requires an explicit `--conf` run id (see *Triggering the DAG* below).

Earlier evidence in `evidence/gcp-composer-20260819/`, `evidence/gcp-composer-20260818/` (first green DAG, plus the three failures only that path exposed), `evidence/gcp-run-20260804/` and `evidence/gcp-composer-20260805/`.

Two things still gate a *fresh* run on a new account:

- **An OPEN billing account.** `gcloud billing accounts list` must show `OPEN: True`; Composer and Kafka will not provision against a closed one.
- **The expensive resources are torn down between runs** (`terraform destroy -target=module.composer`), so a `plan` on this project will propose recreating them.

BigQuery is also reachable on the free sandbox tier, which is why `BQ_TARGET=real` works against a sandbox project (set via `BQ_PROJECT`) at zero cost.

B1. Credentials — user ADC, not a service-account key

```
gcloud auth application-default login     # opens a browser; needs the cloud-platform scope
unset GOOGLE_APPLICATION_CREDENTIALS      # or it overrides the file you just created
```

A skewed clock breaks service-account keys and not user credentials. If `timedatectl` reports `System clock synchronized: no` and the host is more than a few minutes off, every SA-key auth fails with *"Invalid JWT: Token must be a short-lived token (60 minutes)"* — the key signs a JWT locally and Google rejects its `iat` / `exp`. User ADC exchanges a refresh token server-side and is unaffected, so it is the more forgiving path. Fix the clock with `sudo timedatectl set-ntp true` (verify it actually syncs) before using a key.

B2. Manual prologue

Terraform cannot create the project it authenticates into, nor the bucket holding its own state. So:

```
# 1. Reopen a billing account in the console, then confirm it:
gcloud auth login
gcloud auth application-default login
gcloud billing accounts list        # must show OPEN: True

export TF_VAR_billing_account=XXXXXX-XXXXXX-XXXXXX
export TF_VAR_project_id=mig-000001-1-dev
export TF_VAR_region=europe-west1
# export TF_VAR_org_id=...          # if creating inside an organisation
```

B3. Bootstrap, then migrate state

The chicken-and-egg: the `backend "gcs"` block names a bucket that Terraform has not created yet. `-backend=false` is **not** enough on its own — Terraform still sees a backend in the configuration and refuses the apply with *"Changes to backend configurations require reinitialization"*. Comment the block out for the bootstrap, then restore it:

```
cd terraform/envs/dev

# 1. Comment out the `backend "gcs"` block in main.tf (four lines).
#    Point it at your own <project>-tfstate first if you are not using this project —
#    a backend block cannot use variables.

# 2. Project, ~15 APIs and the state bucket, with local state.
terraform init
terraform apply -target=module.bootstrap -var=create_project=false

# 3. Restore the backend block, then copy the local state into the bucket it names.
terraform init -migrate-state
```

`-var=create_project=false` on every command: the project was created by hand (a service account cannot create one under "No organization"), so Terraform adopts it rather than trying to recreate it.

B4. Apply the rest

```
terraform plan -out=tfplan
terraform apply tfplan
```

Defaults are deliberately cheap:

Flag	Default	Why
`enable_composer`	`false`	Composer 2 is **~$300-400/month even idle**
`enable_kafka`	`false`	Managed Kafka bills per vCPU-hour

Turn them on explicitly:

```
terraform apply -var=enable_composer=true -var=enable_kafka=true
```

B5. Artefacts Terraform deliberately does not own

Build outputs change with every commit; coupling them to `terraform apply` couples deploys to infrastructure changes.

```
make seed-secrets      # PGP private key + Target System credentials into Secret Manager
make build-images      # all six images, on one tag (see below)
make deploy-dataform   # push dataform/ to the git remote, or compile it locally if unlinked
make deploy-dags       # rsync composer/dags/ to the Composer DAG bucket
make smoke-gcp         # one tiny end-to-end run on real infrastructure
```

All six images are required, and they must share a tag. Every DAG task is a pod, and the images split by base image: `build_java_images.sh` publishes `loader-app` + `recon-service` (`Dockerfile.javaapp`) and `dataform-runner` (`Dockerfile.dataform`); `make build-templates` publishes `file-processor`, `data-enrichment` and `json-producer` (`Dockerfile.dataflow`). The DAG pins all six with a single `MIG_JAVA_IMAGE_TAG`, so a tag that differs between the two halves means a pod asking for an image that was never pushed — the 2026-08-19 retest failure. Each script derives its tag independently when it runs, so running them separately either side of an edit lands them on `<sha>` and `<sha>-dirty`. `make build-images` derives the tag once and exports it to both; run the scripts individually only with an explicit `MIG_JAVA_IMAGE_TAG` set for both.

What *is* optional is the other half of `build-templates`: the three Flex Template spec JSONs it writes to GCS. The DAG does not read them — it runs those same images as pods that submit their own Dataflow job. The specs are kept for the eventual move back to templates.

Then point the pipelines at the new environment:

```
terraform -chdir=terraform/envs/dev output -raw env_file >> .env
```

B6. Teardown

```
terraform destroy
```

`force_destroy` is set on buckets **only in** `envs/dev`. A production landing bucket holds the migration's evidence and must not be destroyable by a stray command.

B7. The Load lane needs a Target System to post to

Target System belongs to another team and does not exist for this prototype. Locally the podman stack runs `target-system-mock`; on GCP the same mock runs on **Cloud Run** (`module.target_system_mock`, `enable_target_system_mock`, default on — it scales to zero, so an idle environment bills nothing for it).

Build and push it before the first apply, because Cloud Run validates the image reference:

```
REG=${TF_VAR_region}-docker.pkg.dev/${TF_VAR_project_id}/mig-dataflow
podman build -f apps/target-system-mock/Dockerfile -t "$REG/target-system-mock:latest" .
gcloud auth application-default print-access-token \
  | podman login -u oauth2accesstoken --password-stdin ${TF_VAR_region}-docker.pkg.dev
podman push "$REG/target-system-mock:latest"
```

`terraform output -raw env_file` then carries `TARGET_SYSTEM_URL`. Without it the loader falls back to `http://localhost:8080`, which inside a container is nothing at all: every document fails `transport failure` after exhausting its retries, which reads like a data problem and is an infrastructure one.

B8. Managed Kafka — VPC connector, IAM, OAUTHBEARER

The confirmation stream (`docs/PLAN-CHANGES-22082026.md`) needs Managed Kafka reachable over `SASL_SSL` / `OAUTHBEARER` from both the mock (publishing confirmations) and recon (consuming them). Three things the broker needs beyond `enable_kafka=true`, all provisioned by the same apply — none of them is a manual step:

1. **Serverless VPC Access connector** (`module.vpc_connector`, `mig-vpc-connector`). Managed Kafka is VPC-internal (no public endpoint). The Cloud Run mock egresses through this connector to reach the broker — `egress = PRIVATE_RANGES_ONLY` routes only RFC1918 (the broker) through it, everything else stays on the default path. The connector auto-creates its own `/28` subnet (`10.20.128.0/28`, non-overlapping with `mig-subnet`'s `10.20.0.0/20`) so there is no separate subnetwork resource to manage.

2. **`roles/managedkafka.client` — project-level, not cluster-level.** Managed Kafka has no cluster-level IAM resource: the Terraform provider exposes only `cluster`, `topic` and `acl` (verified via `terraform providers schema -json`, not assumed). The client role is a project-level grant via `google_project_iam_member`, conditionally merged into `local.project_roles` in `terraform/modules/iam` when `kafka_cluster_id != ""`. Three principals receive it: `dataflow-`

`worker` (json_producer on the DAG), `recon-service` (consumes confirmations on the DAG) and `target-system-mock` (publishes confirmations from Cloud Run). Without it the OAUTHBEARER handshake authenticates the SA but the broker refuses the connection as unauthorized.

3. **The token is not the access token.** Managed Kafka rejects a bare OAuth access token with *"Authentication failed ... invalid credentials with SASL mechanism OAUTHBEARER"* — an error that names the mechanism, not the encoding. It wants a JWT-shaped, dot-joined, base64url value: `b64({"typ":"JWT","alg":"GOOG_OAUTH2_TOKEN"})` + `.` + `b64({"exp","iat","scope":"kafka","sub":<service-account email>})` + `.` + `b64(<access token>)`. Both sides build it: `ro.mig.common.GcpTokenOauthCallbackHandler` (Java) and `_kafka_token()` in `pipelines/common/sinks.py` (Python), mirroring Google's own `com.google.cloud.hosted.kafka.auth.GcpLoginCallbackHandler`. The `sub` comes from the metadata server's default service account, overridable with `GOOGLE_MANAGED_KAFKA_AUTH_PRINCIPAL` (the same variable Google's handler reads).

4. **kafka-clients must be 4.x on JDK 25.** 3.7's `OAuthBearerSaslClientCallbackHandler` calls `Subject.getSubject(AccessControlContext)`, which JDK 24+ answers with `UnsupportedOperationException: getSubject is not supported`. Every OAUTHBEARER connection then dies mid-handshake and reaches the caller only as `TimeoutException: Timeout expired while fetching topic metadata`.

5. **Keep an SLF4J binder on the classpath.** Without one, kafka-clients prints "Defaulting to no-operation (NOP) logger implementation" and discards its own diagnosis — which is why items 5 and 6 both presented as the same meaningless metadata timeout. `slf4j-simple` is a dependency of `recon-service` and `target-system-mock` for exactly this reason. Pin it to the major version `kafka-clients` brings (`slf4j-api` 1.7 ⇒ `slf4j-simple` 1.7.x); a mismatched binder is silently not a binder.

6. **`make build-images` builds the Cloud Run mock too — check that it still does.** Terraform pins the `target-system-mock` Cloud Run service to `:latest`. Until 2026-08-23 the build script did not list that image at all, so the deployed mock stayed on a months-old build while every other component moved; the confirmation fixes shipped everywhere except the process that publishes confirmations. Two related traps once you are rebuilding it by hand: podman's layer cache will happily reuse a `mvn package` layer across a `pom.xml` change (use `--no-cache` when a dependency version moves), and in **zsh** `"$IMG:latest"` applies the `:l` history modifier and silently pushes a tag named `…mockatest` — write `"${IMG}:latest"`.

7. **`KAFKA_SECURITY_PROTOCOL` has to reach the DAG's *pods*, not just the Composer environment.** A `KubernetesPodOperator` passes only what its `env_vars` names — a Composer `softwareConfig.envVariables` entry is visible to the DAG *file* (which runs in the scheduler) and to nothing else. So the DAG reads `KAFKA_SECURITY_PROTOCOL`, `KAFKA_BOOTSTRAP` and `KAFKA_TOPIC` at parse time and forwards them into both pod flavours: the Beam pods (json_producer's Kafka sink, which ships the config into the Dataflow workers) and the Java pods (recon's confirmation consumer). The Composer module derives the value — `SASL_SSL` whenever `kafka_bootstrap != ""`, `PLAINTEXT` otherwise — so it cannot drift from whether a cluster actually exists. Found and fixed on the 2026-08-23 Heavy loop: without it recon opens a PLAINTEXT connection to a SASL_SSL-only broker and reports zero confirmations instead of failing loudly.

8. **`KAFKA_SECURITY_PROTOCOL=SASL_SSL`** env on the mock. Flips its Java producer from PLAINTEXT (local redpanda) to `SASL_SSL` / `OAUTHBEARER` with the shared `ro.mig.common.GcpTokenOauthCallbackHandler`, which fills the `OAuthBearerTokenCallback` from the same cloud-platform access token the Java/Python GCS clients already use. The Python `json_producer` mirrors this via `confluent_kafka`'s `oauth_cb` (`pipelines/common/sinks.py`), reading `MIG_KAFKA_TOKEN` that `run_pipeline.py`'s `gcp_endpoints()` sets alongside `MIG_GCS_TOKEN` from one `google.auth.default()` call.

Cloud Run v2 schema note. The mock's `service_account` lives inside `template {}` (not at the service top level), and the VPC egress block is `vpc_access` (not `vpc_access_connector`) with `connector` + `egress` fields. Both differ from the plan's original assumptions and from older Cloud Run v1 docs.

smoke-gcp host limitation (the decisive constraint)

`make smoke-gcp` runs `recon-service` and `json_producer` on the **host laptop** (inside the `mig-toolbox` podman container), not in the VPC. The Serverless VPC Access connector serves serverless GCP services (Cloud Run), **not a host laptop** — a VPC connector is not a VPN. So the host-side recon/json_producer **cannot reach VPC-internal Kafka at all**. The only execution path where these run inside the VPC is the Composer DAG (pods on Composer's GKE cluster in `mig-vpc`). A live criterion-9 green on GCP therefore requires the full Composer DAG path (~20-40 min Composer create, ~$300-400/mo idle, RBAC via VM, 6 image rebuilds, DAG trigger, pre-staged extract, poll to green) — not `make smoke-gcp`.

This loop lands the code + infra + `terraform plan` / `apply` validation, not a live criterion-9 run. `make smoke-gcp` with `enable_kafka=true` is still useful: it confirms the broker provisions, the IAM grants bind, and the mock publishes (the mock runs on Cloud Run, inside the VPC). It does not confirm recon consumes — that is the DAG-only path.

Composer — create, cost, teardown

Composer is the one resource whose behaviour surprises people, so it gets its own section.

The control plane is not reachable from your laptop

This invalidates the phase-2 instructions below on any recently created environment. Composer 2 now provisions its GKE cluster with

```
masterAuthorizedNetworksConfig.enabled          = true   # and no CIDRs allowed
privateClusterConfig.privateEndpointEnforcement = true   # public endpoint disabled
```

regardless of `enable_private_endpoint = false` in `terraform/modules/composer` — that setting governs the *environment*, not the cluster's control plane, and Google's defaults have since tightened. So `kubectl get namespaces` times out against the public endpoint (`dial tcp <ip>:443: i/o timeout`), and Terraform's `kubernetes` provider — `module.composer_rbac` — fails for exactly the same reason.

Use `local/scripts/gcp/composer_rbac_via_vm.sh`, which runs the same three objects from a throwaway VM inside the VPC and deletes it afterwards:

```
source local/scripts/gcp/_env.sh
bash local/scripts/gcp/composer_rbac_via_vm.sh
```

It prints the namespace it discovered; pass that to Terraform afterwards so state matches reality. The alternative is to open the control plane to your IP (`--enable-master-authorized-networks --master-authorized-networks=<ip>/32` plus disabling private-endpoint enforcement), which keeps the RBAC in Terraform but widens the cluster's exposure.

It is a two-phase apply — skipping phase 2 is the classic failure

Composer does not expose its Kubernetes namespace as a Terraform attribute, and `module.composer_rbac` is gated on knowing it. If you apply once and stop, the RBAC is **silently skipped** — no error — and then every pod task fails with `pods is forbidden: ... cannot list resource "pods"`.

```
# Phase 1 — create the environment (~$300-400/mo, ~20-40m to create).
terraform -chdir=terraform/envs/dev apply -var=enable_composer=true \
  -var=create_project=false -var=java_image_tag="$(bash -c 'source
local/scripts/gcp/_env.sh; derive_image_version')"

# Phase 2 — the Role, the RoleBinding and the three KSAs, applied from inside the VPC
# because the cluster's control plane is not reachable from here (see above). It prints
# the namespace it discovered.
bash local/scripts/gcp/composer_rbac_via_vm.sh

make deploy-dags
```

Phase 2 is the script, not a second `terraform apply`. Running the apply with `-var=composer_pod_namespace=…` is what the section above warns about, and it fails exactly as described — the Google-side Workload Identity bindings are created, then every `kubernetes_*` resource times out:

```
Error: Post "https://<control-plane-ip>/apis/rbac.authorization.k8s.io/v1/namespaces/
       composer-2-9-7-airflow-2-9-3-…/roles": dial tcp <ip>:443: i/o timeout
```

That is not a transient failure and retrying does not help. Confirmed again on 2026-08-20. The consequence is that those three Kubernetes objects live outside Terraform state: they exist on the cluster, `terraform plan` still proposes creating them, and they disappear with the environment when Composer is destroyed. Passing `composer_pod_namespace` is worth it only if you have first opened the control plane to your IP.

The SSD quota ceiling that stops the DAG dead

On 2026-08-19 every `KubernetesPodOperator` task failed with *"Pod took longer than 900 seconds to start"*, which reads like a slow image pull and is not. The Kubernetes events tell the real story:

```
FailedScheduling  0/5 nodes are available: 2 Insufficient cpu, 5 Insufficient memory
FailedScaleUp     Node scale up in zones europe-west1-c failed: GCE quota exceeded
```

The exhausted quota is **`SSD_TOTAL_GB`, whose default is 500 GB** — and Composer 2's own Autopilot cluster runs 5 nodes at 100 GB each, consuming all of it. Task pods then need a sixth node that can never be created.

Two things make this hard to see:

- `gcloud compute instances list` returns **nothing**, because Composer 2's Autopilot nodes live in a Google-managed tenant project — while their disks still count against *your* project's quota. `gcloud container clusters describe … --format='value(currentNodeCount)'` shows the 5.
- The message says "GCE quota exceeded", which sends you to CPU. CPU was 10/200. Check `gcloud compute regions describe <region> --format='value(quotas)'` and look for any metric at 100%, not the one you expect.

Shrinking the environment does not help — `workloads_config` is already at the 0.5 CPU / 2 GB floor; the nodes are Autopilot's own overhead. **Request an `SSD_TOTAL_GB` increase for the region before running the DAG on a fresh project**, or expect every pod task to time out.

Two things only the DAG path reveals

Both of these passed every local and `smoke-gcp` test and failed on the first real DAG run, because the DAG is the only path where the pods run as the **least-privilege `dataflow-worker` service account** via Workload Identity. Everywhere else the code authenticates as the human operator, whose rights are far broader.

- **`storage.buckets.create` is not granted, and should not be.** `LoaderApp` and `ExtractorApp` call `store.createBucket(...)` at startup — a convenience for fake-gcs-server, which starts empty. On real GCS the buckets are Terraform-owned and the call returns `403 … does not have storage.buckets.create access`, failing the task. `HttpObjectStore.createBucket` now returns immediately against `storage.googleapis.com`; provisioning belongs to Terraform there.
- **A rebuilt image under the same tag does not take effect.** `build_java_images.sh` tags with the git SHA plus `-dirty` on an unclean tree, so the *same* tag is reused for every rebuild from that tree — while Kubernetes defaults to `IfNotPresent` for any tag other than `:latest`. A node that already pulled the tag keeps serving the old layer, so a fix appears not to work. The DAG now sets `image_pull_policy="Always"` on all pod tasks.

Triggering the DAG — the run id is a `--conf` argument, not the Airflow run id

Two things about `mig_000001_1_migration` that `make smoke-gcp` never exercises, because `run_pipeline.py` generates its own run id and runs the extractor in-process:

- **The DAG does not run the harness or the extractor.** Its first task is a `GCSObjectExistenceSensor` on `extraction/{RUN_ID}/ACCOUNT.FLG` (12h timeout, `mode="reschedule"`). The extract must be **pre-staged**: run the extractor-app on the host pointing at the real landing bucket with the same `--run-id`, so the sensor finds the `.FLG` semaphore and the downstream T+L+R pods proceed. `RUN_ID` is the only thing that scopes a run (one full snapshot — D5), so the pre-staged extract and the DAG run must share it exactly.
- **A bare `dags trigger` produces a run id the pods reject.** `RUN_ID` defaults to `{{ dag_run.conf.get('run_id', run_id) }}` — the Airflow run id when `--conf` is absent. Airflow's default is `manual__2026-08-22T14:11:17+00:00`, which contains `:` and `+` and fails `SAFE_IDENTIFIER = ^[A-Za-z0-9_.-]+$` — the guard every pipeline module enforces on `--run-id` (`file_processor`,

`data_enrichment`, `json_producer`, `ReconService`). Every pod raises `ValueError` before it reads a record. And because `max_active_runs=1`, that doomed run also blocks any new trigger until its sensor times out 12h later.

Always trigger with an explicit safe-identifier `--conf`, and pre-stage the extract first:

```
RUN_ID=run-dag-20260822-1411
# 1. pre-stage the extract on the host (harness + extractor-app → real landing bucket)
. .venv/bin/activate
python -m harness.generate --accounts 50 --format copybook
java -jar apps/extractor-app/target/extractor-app.jar \
  --input local/data/mainframe/ACCOUNT.src \
  --bucket "$(grep GCS_LANDING_BUCKET .env | cut -d= -f2)" \
  --gcs-host https://storage.googleapis.com --run-id "$RUN_ID" \
  --gnupg-home local/keys --recipient "$(grep PGP_RECIPIENT .env | cut -d= -f2)" --split 300
# 2. trigger the DAG with that run id
gcloud composer environments run mig-composer --location=europe-west1 \
  --project="$TF_VAR_project_id" dags trigger -- \
  mig_000001_1_migration --conf "{\"run_id\":\"$RUN_ID\"}"
```

If a bare-trigger run is already queued, clear it before re-triggering — `dags list-runs -d mig_000001_1_migration` to confirm, then `dags delete mig_000001_1_migration` (the DAG file stays in the bucket and re-imports; only the run history is cleared).

The DAG's environment variables belong to Terraform

The DAG reads its configuration from the Composer environment, not from `.env`, and **Terraform owns `softwareConfig.envVariables`**. Setting them with `gcloud composer environments update` appears to work and is then silently reverted by the next apply — which is how a hand-set `MIG_TARGET_SYSTEM_URL` vanished mid-session and the loader started failing with `URI with undefined scheme` three DAG runs later, long after the change that caused it.

They are declared in `terraform/modules/composer` and passed from `envs/dev/main.tf`. The only one you supply per deploy is the image tag:

```
terraform -chdir=terraform/envs/dev apply -var=create_project=false \
  -var=enable_composer=true -var=java_image_tag=<tag make build-images printed>
```

`java_image_tag` matters: by default the script tags with the git SHA, plus `-dirty` on an unclean tree — and `<sha>-dirty` is reused by every rebuild from that tree, so it stops identifying the contents. Set `MIG_JAVA_IMAGE_TAG` to pin an explicit tag and pass the same value to Terraform:

```
MIG_JAVA_IMAGE_TAG=my-tag make build-images
terraform -chdir=terraform/envs/dev apply … -var=java_image_tag=my-tag
```

`make build-images` prints the tag it used at the end, and passes the same value to both build scripts — which is what keeps the Java images and the Flex Template images findable under the one tag Terraform hands the DAG.

Two further traps. **`GCP_PROJECT` is reserved** — Composer sets it, and including it rejects the whole update with *"Environment variables [GCP_PROJECT] may not be overridden"*. And an environment update takes **several minutes**, longer than most CLI timeouts; the operation continues server-side, so poll `describe --format='value(state)'` rather than re-running it. While it reports `UPDATING`, `gcloud composer environments run` refuses to execute at all.

Why creation takes ~20–40 minutes

Composer is not "one VM" — it is a managed GKE Autopilot cluster + a Cloud SQL Airflow database

- a web server + schedulers/workers + a DAG bucket + IAM wiring, provisioned sequentially on a fresh project:

1. **Enable the GKE API** (~1–2m).
2. **Provision a GKE Autopilot cluster** (~10–15m) — Composer creates its own private cluster; Autopilot spins node pools, networking, the control plane.
3. **Provision the Airflow Cloud SQL DB** (~3–5m) — a dedicated Postgres instance for metadata.
4. **Spin up the Airflow control plane** (~5–10m) — web server, scheduler(s), triggerer, workers pulled as containers, plus healthchecks.
5. **Run Airflow DB migrations + healthchecks** (~3–5m) before the environment reports healthy.

A subsequent `apply` that only touches an existing environment is minutes, not tens of minutes. There is no "small" tier that skips the cluster — the GKE cluster and the DB exist even when idle, which is exactly why Composer bills **~$300–400/month from creation**, not from the first DAG run.

Two cautions while it provisions

- **Don't Ctrl-C the apply.** Terraform would lose track of a half-created environment and you would have to import or hand-delete the tenant resources (GKE cluster, Cloud SQL DB, DAG bucket). Let it finish or fail on its own.
- **`gcloud sql instances list` returns nothing.** That is not a failure — Composer's metadata database lives in a Google-managed tenant project, so it is invisible from yours. Don't read its absence as a stall.

Turning the expensive parts off again

```
terraform -chdir=terraform/envs/dev destroy -target=module.composer
terraform -chdir=terraform/envs/dev destroy -target=module.kafka
```

Scoped on purpose: an unscoped `destroy` takes the buckets and datasets with it. Composer and Kafka are the only two resources that bill meaningfully while idle.

What actually changes between A and B

Unchanged — the payoff for the adapter layers

`dataform/definitions/*.sqlx` · `contracts/tds/*.def` · `contracts/mappings/*.yaml` · `contracts/schemas/*.json` · the two-door and balancing logic · Beam `DoFn` transform bodies · Java app business logic.

If any of these needed editing at cutover, an adapter boundary was drawn in the wrong place.

Config flips

Area	Local	Real GCP
Beam runner	`DirectRunner`	`DataflowRunner` + region, temp/staging locations, service account, subnetwork, `--no_use_public_ips`
BigQuery sink	`InsertAllBigQueryWriter` — the emulator implements no load jobs	`FileLoadsBigQueryWriter` — NDJSON staged in GCS, then a load job. Writes below its threshold still stream, because load jobs are capped per table per day
Object storage	`STORAGE_EMULATOR_HOST`	unset it; real `gs://`, ADC credentials
Identity	none	one least-privilege service account per component

All four are already routed through `pipelines/common/config.py` and `pipelines/common/storage.py`.

Genuine implementation work

1. **Flex Templates** — built and published (`make build-templates`, `Dockerfile.dataflow`, spec JSONs in GCS), but the DAG does **not** launch them as Flex Templates today: a Flex Template launch *stages* the job rather than running it, so `wait_until_finish()` and the metrics read have no job to query. The DAG runs each pipeline module in a `KubernetesPodOperator` pod that submits to Dataflow and blocks on the result. The template path is kept as the destination once the pipelines are lifecycle-agnostic and read through `ReadFromBigQuery` rather than `beam.Create`.
2. **Dataform repository** — the Dataform repo is **unlinked** (no git remote), so the DAG does *not* use `DataformCreateCompilationResultOperator`: that operator needs `git_commitish`, and every compile failed with `400 The git reference 'main' could not be resolved`. Instead `dataform_run` is a pod built from `Dockerfile.dataform` running the same `dataform compile --json` + `local/scripts/run_dataform.py` executor that runs locally. Linking a git remote and switching to the operators is the remaining work.
3. **Composer operators** — the DAG is GCP-only; the old `MIG_EXECUTION_MODE` dual mode is gone. `composer/dags/mig_000001_1.py` runs the three Beam pipelines and the two Java apps as `KubernetesPodOperator` pods on Composer's own GKE cluster (the Beam pods submit to Dataflow; the Java pods run the loader and recon). The local emulator stack is driven by `local/scripts/run_pipeline.py`, not the DAG — `make run-initial` is the local entry point, the DAG is the production one.
4. **Kafka auth** — redpanda PLAINTEXT → Managed Kafka with SASL_SSL/OAUTHBEARER.

5. **Networking** — VPC with Private Google Access, Cloud NAT, and the `tcp:12345-12346` inter-worker firewall rule. Omitting that rule is the most common cause of a Dataflow job that starts and then silently never progresses.

Production concerns that only exist on GCP

Partitioning and clustering at 1.7M accounts / 20B transactions, Dataflow and BigQuery quotas, monitoring the balancing equation, CMEK / VPC-SC / residency, masking at ingest, and reconciliation by `account_key` at volume are all real work that the local stack cannot surface. They are enumerated, sequenced and sized in `production-readiness.md`.

Troubleshooting

Symptom	Cause	Fix
`permission denied ... /var/run/docker.sock`	`docker-compose` picked up instead of podman	use `make up`; it forces podman-compose
`creating a temporary directory: no such file`	podman storage tree incomplete	`mkdir -p ~/.local/share/containers/storage/tmp`
`address already in use` on `make up`	containers from a previous attempt	`make down`, or `podman rm -f $(podman ps -aq --filter name=mig)`
BigQuery client retries for 600s then `RetryError`	emulator returns 500 "already created" instead of 409	handled in `BigQuery._ensure`; do not use `exists_ok=True` against the emulator
Dataflow job starts and never progresses	missing inter-worker firewall rule	`modules/network` provisions `tcp:12345-12346`
`no .FLG semaphore`	the extract is absent or incomplete	check the extractor ran for that run id; the semaphore is written last by design
`accepted=0 duplicates=400` after a re-run	a long-running Target System mock still holds idempotency state from the previous run	`podman restart mig-000001-1_target-system-mock_1` — the mock is behaving correctly
images fail to pull on a fresh machine	`docker.io` is not an unqualified-search registry	add it to `~/.config/containers/registries.conf`
DAG pods raise `ValueError` on `--run-id` before reading a record; new triggers stay queued	a bare `dags trigger` (no `--conf`) made `RUN_ID` default to the Airflow run id `manual__…T…:…+…`, which fails `SAFE_IDENTIFIER`; `max_active_runs=1` then blocks re-triggering	clear the doomed run (`dags delete mig_000001_1_migration`) and re-trigger with `--conf '{"run_id":"<safe-id>"}'` — see *Triggering the DAG* above
DAG sensor waits on `extraction/<run_id>/ACCOUNT.FLG` forever	the DAG is sensor-first — it does not run the extractor; the extract was not pre-staged under that run id	run the extractor-app on the host with the same `--run-id` before (or just after) triggering